PROYECTO DE CLASE - DP-800 SQL AI DEVELOPER ASSOCIATE - NIVEL 3

Mini-SCADA inteligente con SQL Server 2025

Alarmas, búsqueda semántica y RAG con un modelo de lenguaje local

Autor	Marco Martínez del Álamo
Asignatura	Big Data (Tajamar)
Fecha	Octubre de 2026
Repositorio	github.com/MartinezDelAlamoMarco/dp800-mini-scada

2. Objetivo del proyecto

El objetivo es construir, en una sola base de datos de SQL Server 2025, un SCADA pequeño que guarde lecturas de sensores, alarmas, incidencias y manuales, y que además permita preguntar en lenguaje natural. Elegí el Nivel 3 del enunciado, que pide diseño de base de datos y T-SQL, seguridad, optimización, embeddings con búsqueda vectorial y un RAG con un modelo local.

Todo funciona en mi portátil con Windows 11, sin servicios en la nube: SQL Server 2025, SSMS, Ollama para los modelos y nginx como proxy HTTPS. El código completo está en el repositorio de GitHub y aquí solo se enseñan los fragmentos más importantes.

3. Caso de uso

Trabajé en una empresa de IoT dedicada a edificios inteligentes y he usado sistemas SCADA, así que el caso me resulta cercano. Un edificio tiene zonas (recepción, sala de servidores, oficinas, almacén), cada zona tiene nodos IoT y cada nodo tiene sensores de temperatura, humedad, CO2, presión o consumo. Un sensor manda una lectura cada cinco minutos y, cuando algo se sale de lo normal, se abre una alarma que alguien resuelve y anota como incidencia.

El problema es que ese conocimiento queda disperso. Cuando salta una alarma nueva, el técnico no sabe si ya pasó antes ni dónde está el procedimiento, y buscar por palabras exactas no sirve: "el sensor de CO2 marca siempre lo mismo" no contiene la palabra "planas" del manual. Con embeddings, SQL Server encuentra las alarmas y los manuales parecidos por significado, y con RAG un modelo local redacta la respuesta usando solo esos manuales.

4. Arquitectura

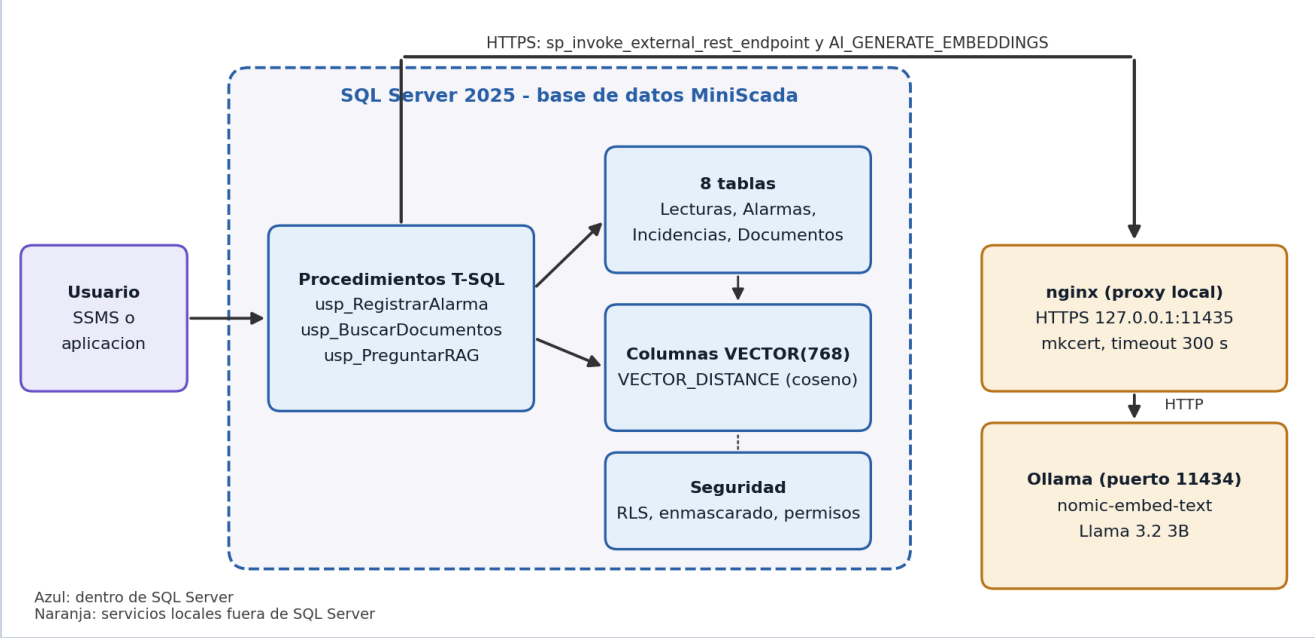


Figura 1. Arquitectura: SQL Server llama por HTTPS a nginx, que reenvía a Ollama.

Las decisiones técnicas más importantes fueron estas:

- **Todo dentro de la base de datos.** Los embeddings se generan con `AI_GENERATE_EMBEDDINGS` y se guardan en columnas `VECTOR(768)`, y la búsqueda es un `ORDER BY VECTOR_DISTANCE`. No hace falta un servicio de

búsqueda aparte.

- **Un proxy HTTPS en medio.** SQL Server 2025 solo llama a endpoints HTTPS y Ollama sirve en HTTP, así que puse nginx con un certificado local de mkcert. Los certificados no están en el repositorio.
- **Dos modelos en Ollama.** `nomic-embed-text` (768 dimensiones) para los embeddings y Llama 3.2 3B cuantizado (Q4_K_M, unos 1,9 GB) para las respuestas, porque cabe en un portátil sin GPU.
- **La generación va por REST.** SQL Server solo admite modelos externos de tipo embeddings, así que el texto del RAG se genera llamando a `/api/chat` con `sp_invoke_external_rest_endpoint`.

5. Modelo de datos

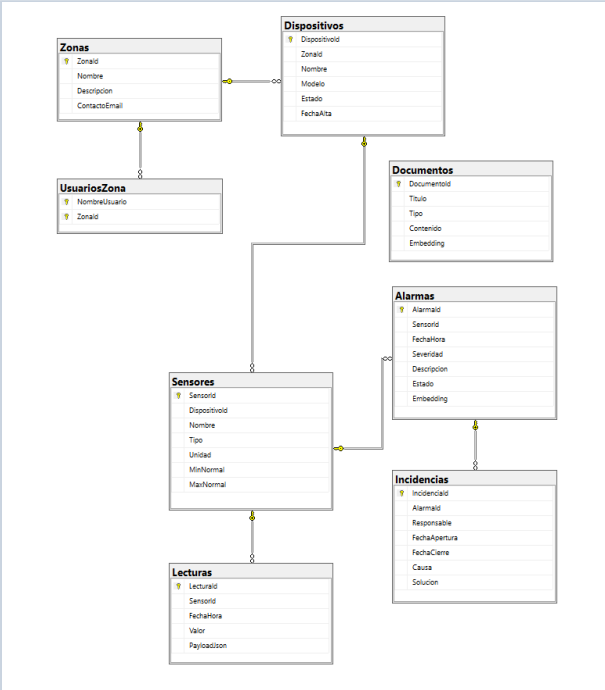


Figura 2. Diagrama de la base de datos MiniScada.

La base de datos tiene 8 tablas organizadas como el edificio: una zona tiene dispositivos, un dispositivo tiene sensores, y cada sensor genera lecturas y puede disparar alarmas. Cada alarma puede tener una incidencia con su causa y su solución.

Tabla	Contenido
Zonas	4 áreas, con un email de contacto (enmascarado)
Dispositivos	8 nodos IoT
Sensores	18, con su rango normal
Lecturas	unas 518.000, con el JSON original
Alarmas	11, con embedding
Incidencias	9
Documentos	10 manuales, con embedding
UsuariosZona	qué zonas ve cada usuario

Las restricciones evitan datos inválidos: claves foráneas, `CHECK` en el rango de cada sensor (mínimo menor que máximo), en las fechas de las incidencias, en la severidad y el estado, y en que `PayloadJson` sea un JSON válido. `Lecturas` es ligera a propósito y al principio solo tiene la clave primaria, para poder medir la optimización.

Los datos los genera un script determinista (`02_datos.sql`): una lectura cada cinco minutos durante 100 días, con un ciclo diario y una variación que sale de una función hash y no de números aleatorios, así que cualquiera obtiene los mismos datos. Añade 11 anomalías a propósito, como dos sensores de CO2 que se quedan fijos o una subida de temperatura en la sala de servidores tras un corte de luz.

6. Funcionalidades implementadas

Bloque	Qué cubre	Scripts	Hecho
A	Diseño de la base de datos y T-SQL: tablas con restricciones, función, 2 vistas, 2 procedimientos, trigger, CTE, funciones de ventana y JSON	01 a 05	Sí
B	Documentación del uso de IA: una ficha por cada parte importante, con mis dudas, lo que acepté, lo que corregí y cómo lo comprobé	fichas	Sí
C	Seguridad: seguridad por filas, enmascarado de datos y permisos mínimos	06, 07	Sí
D	Optimización de una consulta con un índice, midiendo antes y después	08	Sí
E	Embeddings generados dentro de SQL Server en columnas VECTOR(768)	09, 10	Sí
F	Búsqueda semántica con distancia coseno	11	Sí
G	RAG con un modelo local, comparado con el modelo sin documentos	12, 13	Sí

Además, `tests/tests.sql` comprueba de una vez 18 cosas (estructura, datos, restricciones, trigger, seguridad, índice, embeddings y búsqueda) y deja una tabla con OK o FALLO. Las pruebas que tocan datos se hacen dentro de una transacción que se deshace.

7. Consultas y código más importante

Sensores "planos" con una CTE

Dos sensores de CO2 se quedan en un valor fijo durante horas, pero ese valor está dentro del rango normal, así que no se detectan mirando solo el rango. La CTE agrupa por hora y busca las horas con las 12 lecturas y desviación cero:

```
WITH PorHora AS (
    SELECT SensorId,
           DATEADD(HOUR, DATEDIFF(HOUR, 0, FechaHora), 0) AS Hora,
           COUNT(*) AS Lecturas, STDEV(Valor) AS Desviacion
    FROM dbo.Lecturas
    GROUP BY SensorId, DATEADD(HOUR, DATEDIFF(HOUR, 0, FechaHora), 0)
)
SELECT d.Nombre AS Dispositivo, s.Nombre AS Sensor, MIN(p.Hora) AS Desde, COUNT(*) AS HorasPlanas
FROM PorHora p
JOIN dbo.Sensores s ON s.SensorId = p.SensorId
JOIN dbo.Dispositivos d ON d.DispositivoId = s.DispositivoId
WHERE p.Lecturas >= 12 AND p.Desviacion = 0
GROUP BY d.Nombre, s.Nombre;
```

Seguridad por filas

```
CREATE SECURITY POLICY Seguridad.PoliticaZonas
    ADD FILTER PREDICATE Seguridad.fn_SensorPermitido(SensorId) ON dbo.Alarmas,
    ADD BLOCK PREDICATE  Seguridad.fn_SensorPermitido(SensorId) ON dbo.Alarmas AFTER INSERT,
    ...
WITH (STATE = ON);
```

Búsqueda semántica y RAG

El procedimiento `usp_PreguntarRAG` convierte la pregunta en un vector, recupera los dos documentos más cercanos, monta el prompt y llama al modelo:

```
DECLARE @Vector VECTOR(768) = AI_GENERATE_EMBEDDINGS(@Pregunta USE MODEL ModeloEmbeddings);

SELECT @Contexto = STRING_AGG(CONCAT(N'[', Titulo, N'] ', Contenido), CHAR(10) + CHAR(10))
FROM (SELECT TOP (@Cantidad) Titulo, Contenido FROM dbo.Documentos
      WHERE Embedding IS NOT NULL
      ORDER BY VECTOR_DISTANCE('cosine', Embedding, @Vector)) AS d;

SET @Sistema += N' Usa SOLO la informacion de los documentos que se te dan. Si la respuesta
                   no esta en los documentos, responde exactamente: No lo encuentro en los
manuales.';
-- ... se monta el JSON con STRING_ESCAPE y se llama a Ollama:
EXEC sp_invoke_external_rest_endpoint @url = N'https://localhost:11435/api/chat', @method = 'POST',
     @payload = @Payload, @timeout = 230, @response = @Respuesta_Json OUTPUT;
```

8. Seguridad y optimización

Seguridad por filas, enmascarado y permisos

Hay dos roles: `rol_admin_scada`, que lo ve todo y puede ver los datos sin enmascarar, y `rol_tecnico`, que solo ve las zonas que le asigna `UsuariosZona`. Los usuarios se crean sin login para no poner contraseñas en los scripts. Con la misma consulta, el administrador ve las 11 alarmas y el técnico de la sala de servidores solo 3:

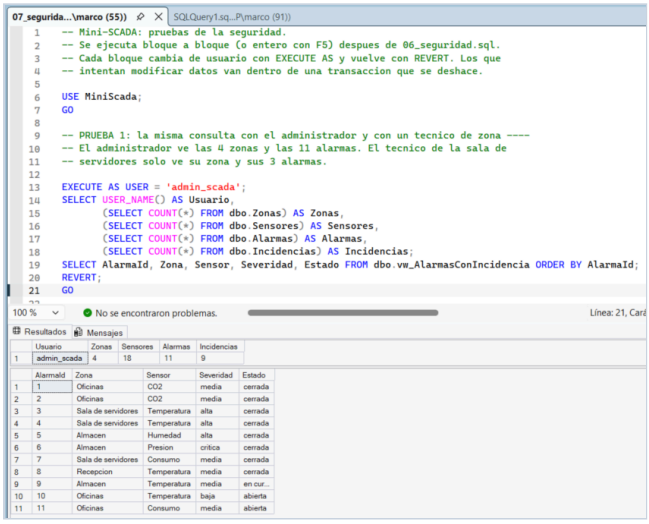


Figura 3. Administrador (EXECUTE AS).

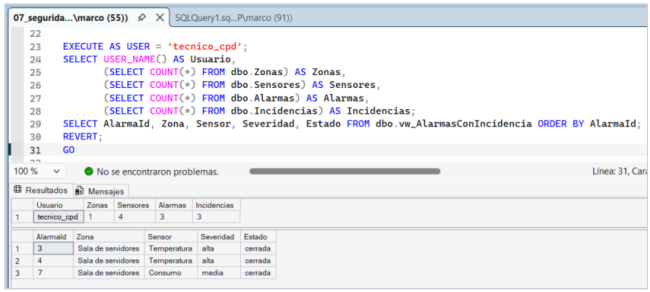


Figura 4. Técnico de la sala de servidores.

El email de contacto de cada zona está enmascarado con `email()`: el administrador lo ve completo y el técnico ve algo como `rXXX@XXXX.com`. Además, el técnico tiene `DENY` sobre la tabla `Lecturas` (ve las lecturas solo a través de una vista que hereda la seguridad por filas) y no puede cambiar la estructura de la instalación ni borrar alarmas.

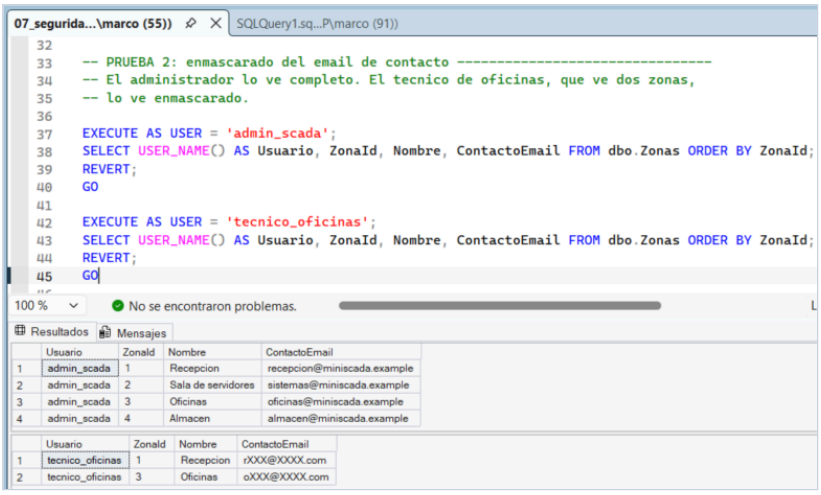


Figura 5. Email de contacto: completo para el administrador y enmascarado para el técnico.

Optimización de una consulta

La consulta calcula la media de CO2 de un sensor durante una semana (2.016 lecturas) sobre una tabla de más de 500.000 filas que solo tenía la clave primaria. El plan inicial hacía un `Clustered Index Scan`. Creé un índice cubriente sobre `(SensorId, FechaHora)` que incluye `Valor` y el plan pasó a un `Index Seek`:

	Sin índice	Con índice
Operador	Clustered Index Scan	Index Seek
Lecturas lógicas	15.771	13
Tiempo con caché fría	241 ms	4 ms
Resultado (media de CO2)	619,98 ppm	619,98 ppm

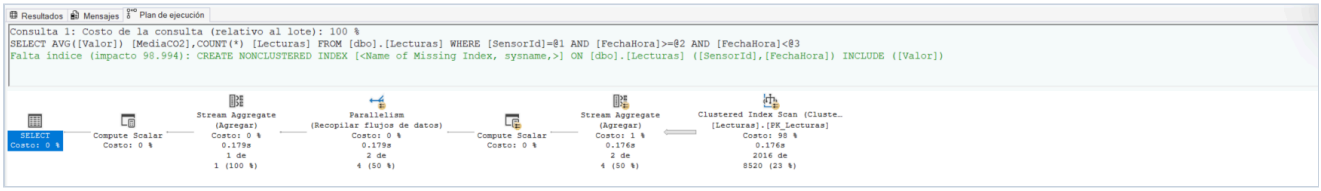


Figura 6. Plan antes del índice: Clustered Index Scan sobre Lecturas.

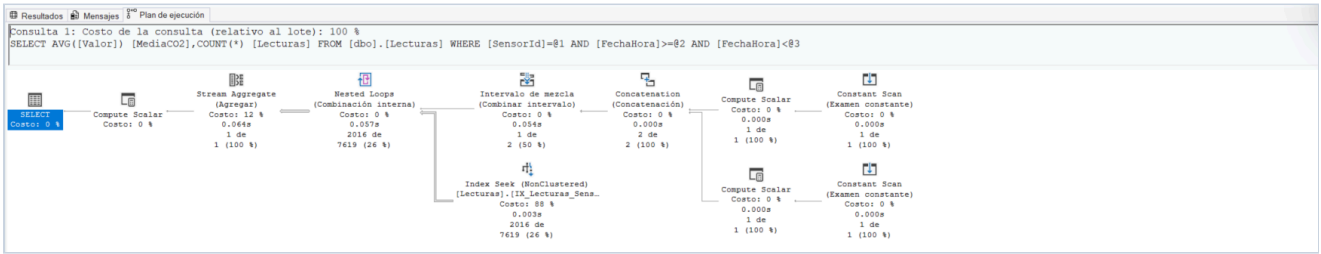


Figura 7. Plan después del índice: Index Seek.

El índice ocupa unos 16,7 MB, que es lo que cuesta. Lee unas 1.200 veces menos páginas, y la diferencia seguirá creciendo porque la tabla crece cada cinco minutos por sensor.

9. Funcionamiento de la IA

Embeddings. Las columnas `Embedding VECTOR(768)` de `Alarmas` y `Documentos` se rellenan con un `UPDATE` que llama a `AI_GENERATE_EMBEDDINGS` con el modelo externo `ModeloEmbeddings` (`nomic-embed-text` en Ollama).

En los documentos se usa el título más el contenido. En total son 21 vectores de 768 dimensiones.

```sql
-- Mini-SCADA: generar los embeddings de las alarmas y de los documentos.
-- Necesita Ollama y nginx en marcha y el modelo externo ModeloEmbeddings creado
-- (ver setup/00_verificar_entorno.sql). Se puede repetir sin problema: solo
-- rellena las filas que todavia no tienen embedding.

USE MiniScada;
GO

-- Alarmas: el texto que se convierte en vector es la descripcion
UPDATE dbo.Alarmas
SET Embedding = AI_GENERATE_EMBEDDINGS(Descripcion USE MODEL ModeloEmbeddings)
WHERE Embedding IS NULL;
GO

-- Documentos: titulo y contenido juntos, para que el titulo tambien cuente
UPDATE dbo.Documentos
SET Embedding = AI_GENERATE_EMBEDDINGS(CONCAT(Titulo, N'. ', Contenido) USE MODEL ModeloEmbeddings)
WHERE Embedding IS NULL;
GO
```

Resultados | Mensajes

	Tabla	Filas	ConEmbedding	Dimensiones
1	Alarmas	11	11	768
2	Documentos	10	10	768

Figura 8. Generación de embeddings y vector devuelto.

Métrica. Uso la distancia coseno (`VECTOR_DISTANCE('cosine', ...)`), que compara hacia dónde apuntan los vectores y no su longitud, y es lo que interesa con textos de tamaños distintos. Cuanto más cerca de 0, más parecido. Con diez documentos no creé un índice vectorial porque la búsqueda exacta es instantánea.

Búsqueda. Probé cinco preguntas escritas como las diría un técnico, sin copiar frases de los manuales. Las cinco devuelven como primer resultado el documento esperado, con distancias entre 0,12 y 0,28. En una de ellas (corte de luz y aire de la sala de servidores) el manual de refrigeración queda segundo, detrás del procedimiento de corte de luz.

Resultados | Mensajes

	Id	Pregunta	Primero	Dist1	Segundo	Dist2
1	1	el sensor de CO2 marca siempre el mismo valor	Procedimiento: sensor de CO2 con lecturas planas	0.1219	Manual: calibracion de sensores de temperatura y...	0.3194
2	2	se fue la luz y el aire de la sala de los servidores no vu...	Procedimiento: corte de luz y arranque de equipos	0.2830	Manual: refrigeracion de la sala de servidores	0.3368
3	3	hay mucha humedad en el almacen, creo que hay agua	Guia: humedad alta en el almacen	0.1590	Procedimiento: temperatura baja en el almacen	0.2858
4	4	la bomba no da presion y casi no sale agua	Manual: bomba de agua del almacen, caida de pres...	0.2756	Procedimiento: temperatura baja en el almacen	0.4044
5	5	cuanto tiempo tengo para atender una alarma critica	Guia: severidad de las alarmas y tiempos de respues...	0.2054	Procedimiento: temperatura baja en el almacen	0.3296

	AlarmaId	Descripcion	Severidad	Distancia
1	3	La temperatura de la sala de servidores supera los 3...	alta	0.3349
2	4	Subida de temperatura en la sala de servidores de ...	alta	0.3731
3	5	Humedad por encima del 90 por ciento en el almac...	alta	0.3800

Figura 9. Ranking de la búsqueda semántica.

RAG. El flujo es pregunta, embedding, dos documentos más cercanos, contexto en el prompt y respuesta del modelo, con temperatura 0. La demo de `13_rag_demo.sql` enseña los cuatro pasos con la pregunta "el sensor de CO2 de la oficina marca siempre lo mismo, que hago": recupera el procedimiento del sensor plano (distancia 0,1309) y el manual de calibración (0,3137), y el modelo contesta con los pasos del primero.

	Paso	Texto
1	1. Pregunta	el sensor de CO2 de la oficina marca siempre lo mismo, que hago
2	2. Documento recuperado	Procedimiento: sensor de CO2 con lecturas planas (distancia 0.1309)
3	2. Documento recuperado	Manual: calibracion de sensores de temperatura y humedad (distancia 0.3137)
4	3. Contexto para el modelo	[Procedimiento: sensor de CO2 con lecturas planas] Cuando un sensor de CO2 muestra siempre el mismo valor durante varias
5		horas, aunque la sala se llene o se vacie, lo normal es que este descalibrado o sucio y no que el aire sea constante. P
6		asos: 1) Comparar con el otro sensor de CO2 de la misma planta. 2) Revisar la entrada de aire del sensor y quitar el pol
7		vo con aire comprimido. 3) Hacer la calibracion a aire exterior (400 ppm) siguiendo el manual del fabricante. 4) Si tras
8		calibrar el valor sigue sin moverse, sustituir el sensor. Dejar anotado en la incidencia la causa y el numero de serie
9		del sensor nuevo. [Manual: calibracion de sensores de temperatura y humedad] Los sensores de temperatura y humedad se c
10		alibran cada seis meses. Para la temperatura se compara con un termometro de referencia a 20 grados; para la humedad, co
11		n una camara de sal saturada que da 75 por ciento. Si la desviacion es mayor de 1 grado o de 3 puntos de humedad, ajusta
12		r el offset en el firmware del nodo. Anotar la fecha y el valor del ajuste en la ficha del sensor. Un sensor que queda f
13		uera de tolerancia despues de ajustar se retira y se cambia.
14	4. Respuesta	Según el procedimiento, debes comparar el valor con el otro sensor de CO2 de la misma planta y revisar la entrada de air
15		e del sensor. Si el valor sigue sin moverse, sustituir el sensor. Dejar anotado en la incidencia la causa y el número de
16		serie del sensor nuevo.

Figura 10. RAG paso a paso: pregunta, documentos recuperados, contexto y respuesta.

Para ver cuánto aporta el RAG, hice las mismas cuatro preguntas sin documentos y con ellos:

Pregunta	**Sin RAG**	**Con RAG**
Tiempo para atender una alarma de severidad alta	Se inventa "15-30 minutos"	"Máximo de 1 hora", como dice la guía
Cada cuánto se calibran los sensores	"Cada 3 meses", genérico	Cada seis meses, con termómetro de referencia y cámara de sal
Presión de la bomba por debajo de 1 bar	Consejos generales	Parar la bomba y las causas del manual
Incendio en la sala de servidores (no está en los manuales)	Consejos inventados	"No lo encuentro en los manuales."

10. Capturas de resultados

	Prueba	Resultado
1	Existen las 8 tablas	OK
2	Las columnas Embedding de Alarmas y Documentos son de tipo vector	OK
3	Existen la funcion, las 2 vistas, los 2 procedimientos y el trigger	OK
4	Lecturas tiene mas de 500.000 filas	OK
5	La vista de ultimas lecturas tiene una fila por sensor	OK
6	Toda alarma cerrada tiene una incidencia con fecha de cierre	OK
7	El CHECK rechaza un sensor con el rango normal al reves	OK
8	usp_RegistrarAlarma rechaza un sensor que no existe	OK
9	El trigger cierra la alarma cuando se cierra su incidencia	OK
10	La politica de seguridad por filas esta activa	OK
11	Existe el indice IX_Lecturas_Sensor_Fecha	OK
12	El administrador ve 11 alarmas y el tecnico de la sala de servidores 3	OK
13	El tecnico no puede leer la tabla Lecturas	OK
14	El email de contacto sale tapado para el tecnico y completo para el administrador	OK
15	Los 10 documentos y las 11 alarmas tienen embedding	OK
16	Existen los procedimientos de busqueda semantica y de RAG	OK
17	La busqueda semantica de CO2 plano devuelve su procedimiento	OK
18	La alarma mas parecida a un servidor caliente es de la sala de servidores	OK

	Correctas	Fallidas
1	18	0

Figura 11. Las 18 pruebas de tests.sql: 18 correctas y 0 fallidas.

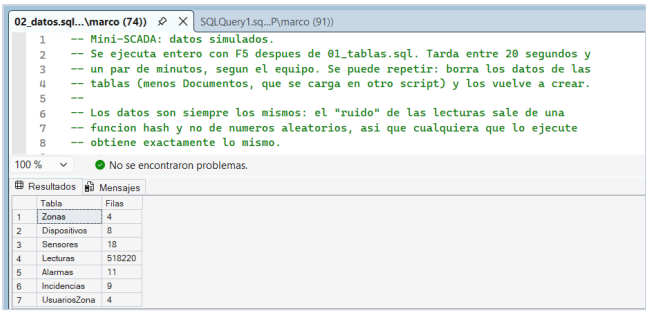


Figura 12. Filas de cada tabla tras cargar los datos.

Resultados Mensajes

	Id	Modo	Texto
5	2	sin RAG	Los sensores de temperatura y humedad se calibran cada 3 meses y se comprueban mediante comparación con sensor
6	2	sin RAG	es de referencia de la misma marca y tipo, o mediante comparación con un calibrador de sensores. También se re
7	2	sin RAG	comienda realizar un calibrado adicional después de cualquier mantenimiento o reparación realizada en el senso
8	2	sin RAG	r. Esto garantiza la precisión y la fiabilidad de los datos proporcionados por los sensores.
9	2	con RAG	Los sensores de temperatura y humedad se calibran cada seis meses, comparándose con un termometro de referenci
10	2	con RAG	a a 20 grados para la temperatura y con una cámara de sal saturada que da 75 por ciento para la humedad. Si la
11	2	con RAG	desviación es mayor de 1 grado o de 3 puntos de humedad, se ajusta el offset en el firmware del nodo. Se anot
12	2	con RAG	a la fecha y el valor del ajuste en la ficha del sensor.
13	3	sin RAG	Si la presión de la bomba del almacen baja de 1 bar, es posible que la bomba no esté funcionando correctamente
14	3	sin RAG	o que haya una pérdida de presión en el sistema. Verifica que la bomba esté funcionando correctamente y que n
15	3	sin RAG	o haya fugas en el sistema. Si el problema persiste, es posible que debas reemplazar la bomba o realizar ajust
16	3	sin RAG	es en el sistema.
17	3	con RAG	Si la presion de la bomba del almacen baja de 1 bar, parar la bomba para no quemar el motor. Causas habituales
18	3	con RAG	son la pérdida del cebado por aire, filtro de entrada obstruido, valvula de aspiración cerrada o junta del ej
19	3	con RAG	e desgastada.
20	4	sin RAG	Si hay un incendio en la sala de servidores, apaga el interruptor de la luz y llama al servicio de emergencia
21	4	sin RAG	inmediatamente. No intentes apagar el incendio tú mismo, ya que esto puede empeorar la situación. Mantén la lí
22	4	sin RAG	nea de comunicación abierta para recibir instrucciones.
23	4	con RAG	No lo encuentro en los manuales.

Figura 13. Respuestas con y sin RAG (preguntas 2 a 4).

Para comprobar que se puede reproducir, borré la base de datos `MiniScada` y ejecuté todos los scripts en orden. Salió sin errores, las 18 pruebas pasaron y tardó unos 2 minutos y 6 segundos, de los que casi 1 minuto y 40 segundos son las ocho respuestas del modelo.

11. Problemas encontrados

- **Que SQL Server hablara con Ollama.** Fue lo que más tiempo me llevó. SQL Server solo llama por HTTPS, así que monté nginx con un certificado de mkcert. El servicio de SQL Server no usa los certificados de mi usuario y hubo que importar la entidad certificadora en el almacén del equipo. Además, `curl` se quejaba de la revocación (se arregla con `--ssl-no-revoke`) y PowerShell estropeaba el JSON, que resolví guardándolo en un archivo.
- **No sabía si la llamada REST había funcionado.** Solo salía "Los comandos se han completado correctamente": la respuesta va por el parámetro `@response OUTPUT` y hay que sacarla con un `SELECT`.
- **El modelo de lenguaje no se descargaba.** `ollama pull llama3.2:3b` fallaba por un error de red. Lo solucioné bajando el mismo modelo desde Hugging Face (`hf.co/bartowski/Llama-3.2-3B-Instruct-GGUF:Q4_K_M`).
- **La generación se cortaba.** nginx cierra la conexión a los 60 segundos y responder tarda más. Subí `proxy_read_timeout` a 300 s.
- **Errores de T-SQL.** Una columna `IDENTITY` de una tabla temporal no se reiniciaba con `DELETE` (con `TRUNCATE TABLE` sí) y `GENERATE_SERIES` exigía que los dos extremos fueran del mismo tipo, pero `LEN` devuelve `BIGINT`.

12. Conclusiones

La parte de IA en SQL Server 2025 es corta de escribir: un `UPDATE` con `AI_GENERATE_EMBEDDINGS` y un `ORDER BY VECTOR_DISTANCE`. Lo difícil fue todo lo de alrededor, sobre todo el proxy HTTPS y los certificados. También me llevo lo que cambia el RAG: sin documentos el modelo se inventa plazos y periodos con total seguridad, y con ellos contesta lo que dicen mis manuales y, si no lo encuentra, lo dice. En un sistema de alarmas eso es lo más importante.

Lo que mejoraría: probar con más documentos y más preguntas (con cientos de manuales sí haría falta un índice vectorial), usar una GPU para probar un modelo mayor, recibir los datos de verdad por MQTT en vez de simularlos, y guardar cada pregunta con sus documentos y su respuesta para revisar después cuándo se equivoca el RAG.